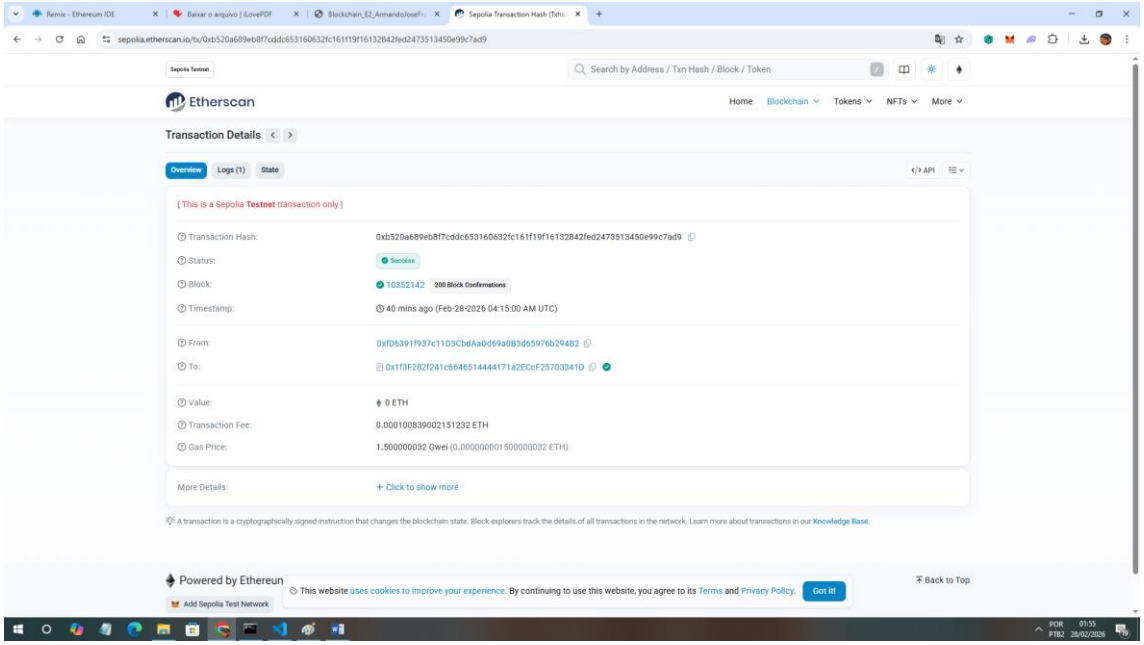
Mini Projeto Web3: Minha Primeira Integração com Blockchain

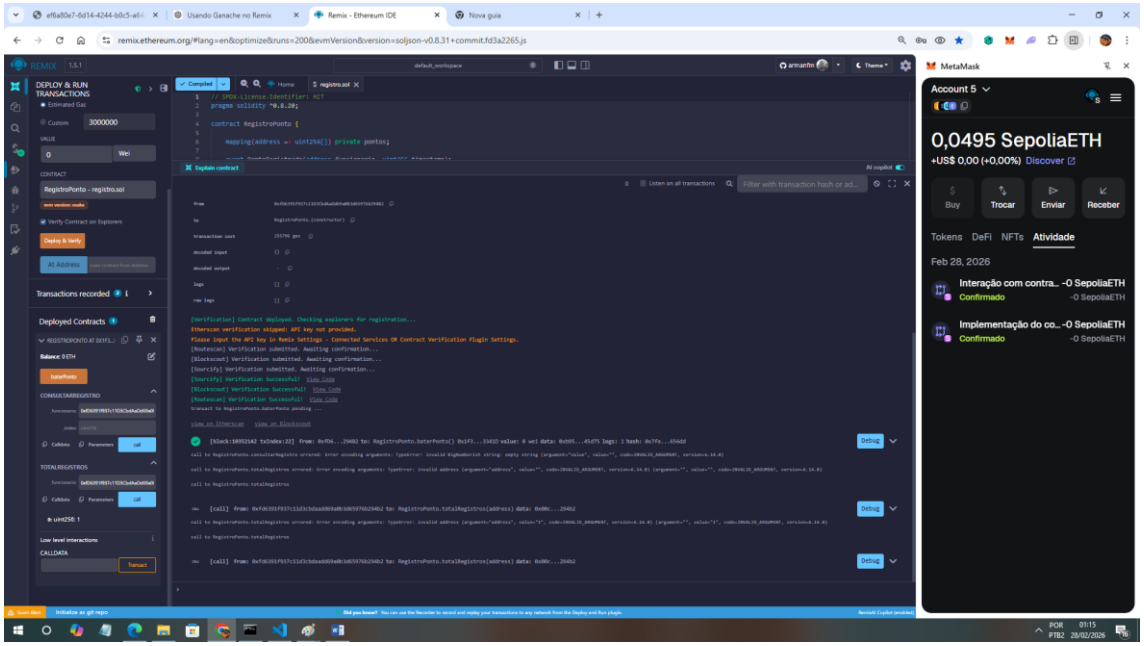
Armando José Freire de Melo

27/02/2026
Turma:1

Desafio 1

Registrar um dado na blockchain





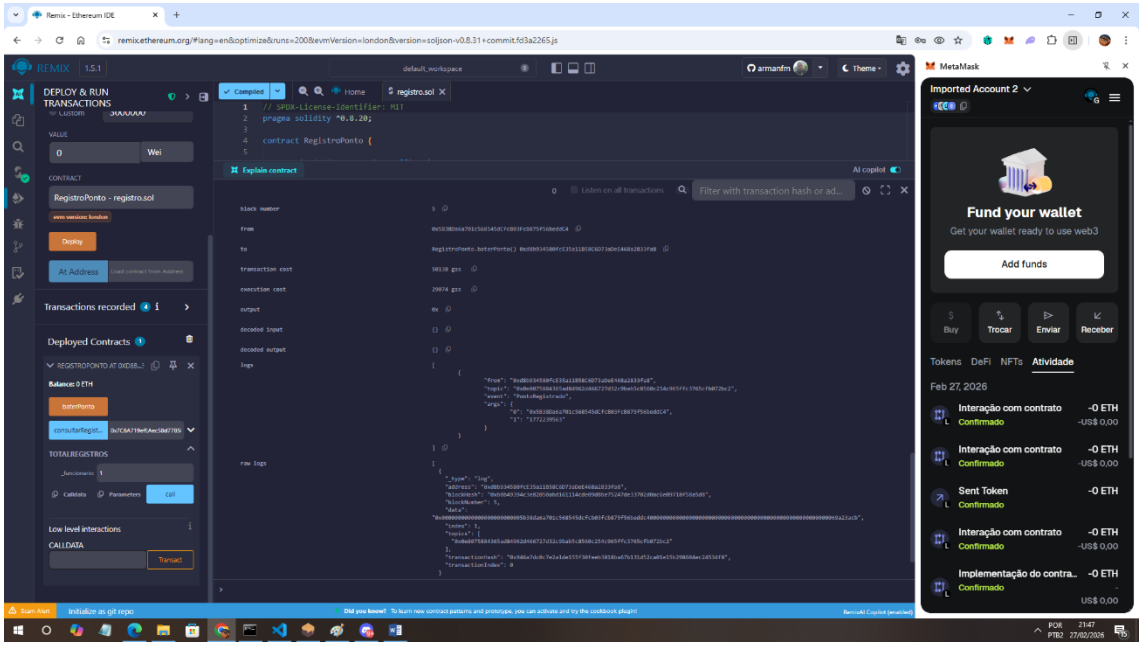
O sistema é baseado conceitualmente em registrar entrada e saída de uma empresa diretamente na blockchain.

Primeiro foi criado o arquivo `registro.sol` contendo o contrato inteligente `RegistroPonto`.

Após criar o arquivo, foi necessário:

1. Compilar o contrato no Remix
2. Ir até a aba Deploy
3. Realizar o deploy na rede (Ganache / testnet local)
4. Confirmar a transação na MetaMask

Somente após o deploy confirmado o contrato ficou disponível para interação.



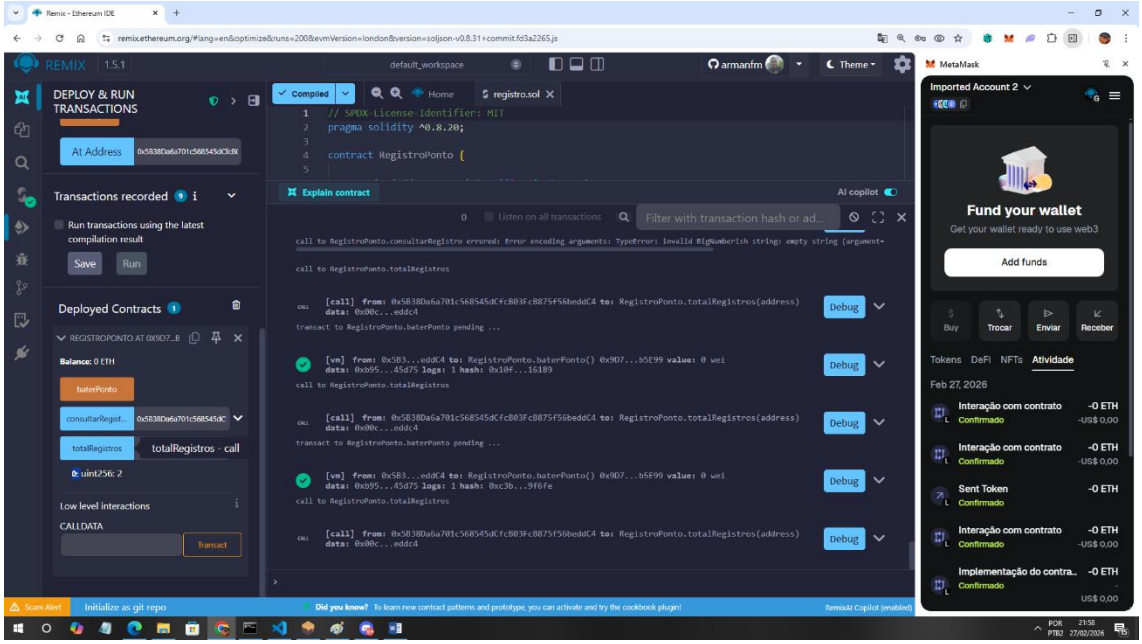
Após a confirmação do deploy, foi possível utilizar o botão `baterPonto()`.

Ao clicar no botão:

- Uma transação é gerada
- A MetaMask solicita confirmação
- O contrato registra o endereço público (`msg.sender`)
- O timestamp da blockchain é armazenado

Também é possível colocar o endereço da chave pública no campo de consulta para verificar os registros.

A função `baterPonto()` pode ser executada várias vezes, simulando múltiplos registros de entrada e saída.



Por fim, é possível consultar quantos registros o funcionário realizou utilizando a função `totalRegistros`.

Na imagem é possível observar que:

- O endereço público foi inserido
- O contrato retornou 2 registros
- Esses registros são imutáveis
- Cada registro possui data e hora (timestamp)

Isso demonstra que o sistema armazena corretamente as entradas e saídas na blockchain.

Observação: para fins de demonstração utilzei o Gamach além do testnet.

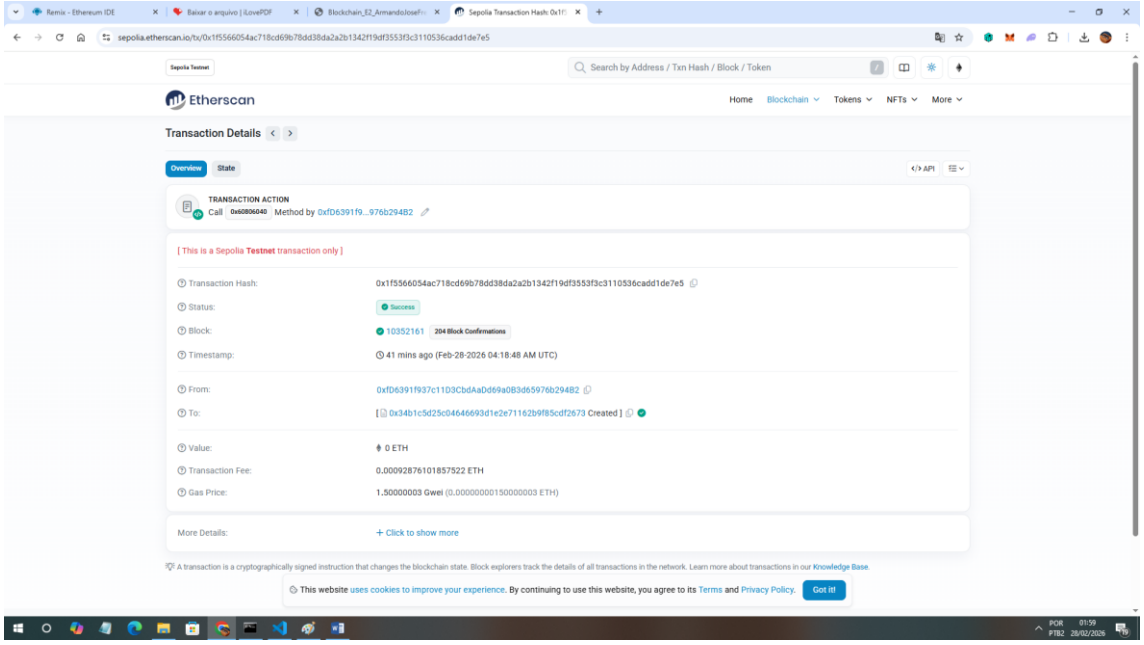
Conclusão 1

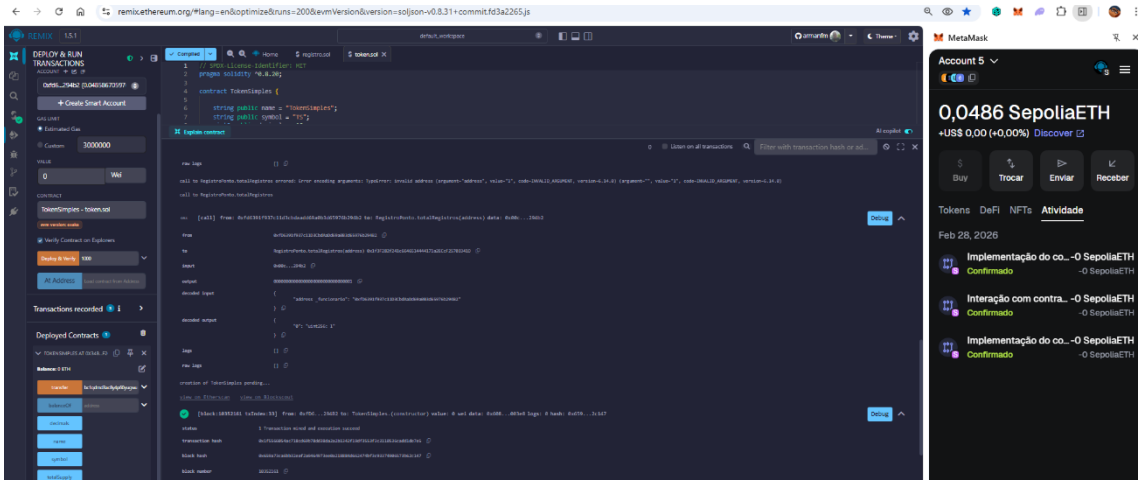
O sistema demonstra que um contrato que pode ser criado e compilado, pode ser implantado na blockchain, que uma função pode ser executada, os dados ficam registrados de forma imutável e é possivel consultar posteriormente.

Criação → Compilação → Deploy → Execução → Registro → Consulta

Desafio 2

Enviar token simples para oura carteira



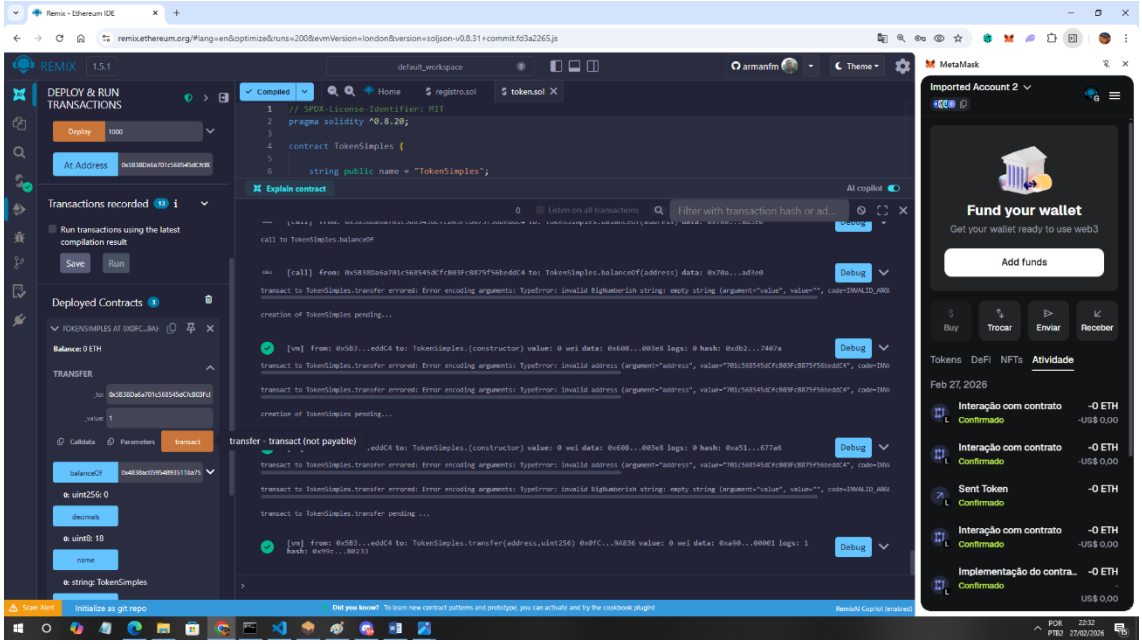


O segundo desafio consiste na criação de um token simples para transações.

Após criar o arquivo `token.sol`, realizamos os seguintes passos:

- Compilamos o contrato.
- Antes do deploy, definimos a quantidade inicial de tokens (supply).
- No campo de deploy, escolhemos o valor inicial do supply.
- No exemplo apresentado, foi definido **1000** como valor inicial.

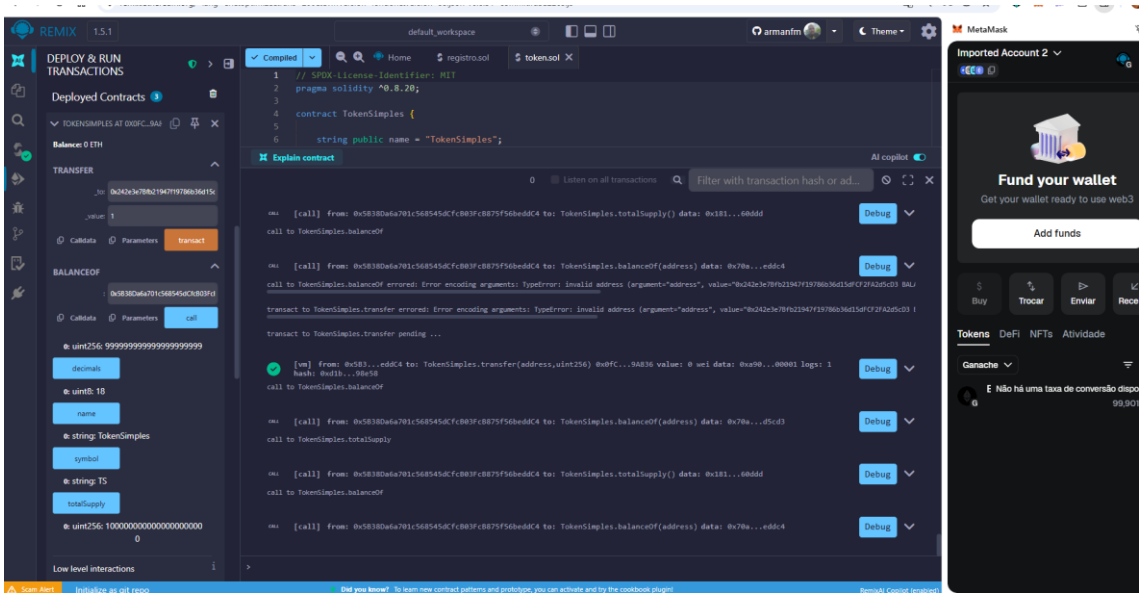
Esse valor será multiplicado automaticamente por `10^decimals` (18 casas decimais), conforme definido no contrato, após definir o valor, realizamos o deploy do contrato.



Após a criação do token:

- Inserimos a chave pública de destino na aba Transfer.
- No campo `value`, informamos a quantidade de tokens que desejamos transferir.
- Executamos a transação clicando em transact.

Essa ação transfere tokens da carteira que criou o supply para outra chave pública.



Por fim, podemos:

- Clicar em totalSupply para visualizar o total de tokens criados.
- Utilizar balanceOf informando uma chave pública específica para verificar o saldo daquela carteira.

Importante destacar:

- A chave pública que realizou o deploy é a detentora inicial de todo o supply.
- Para visualizar a diminuição de tokens, é necessário transferir para outra chave pública.
- Podemos verificar o balanço tanto de quem enviou quanto de quem recebeu.
- Basta utilizar as respectivas chaves públicas no campo `balanceOf`.

Observação: para fins de demonstração utilzei o Gamach além do testnet.

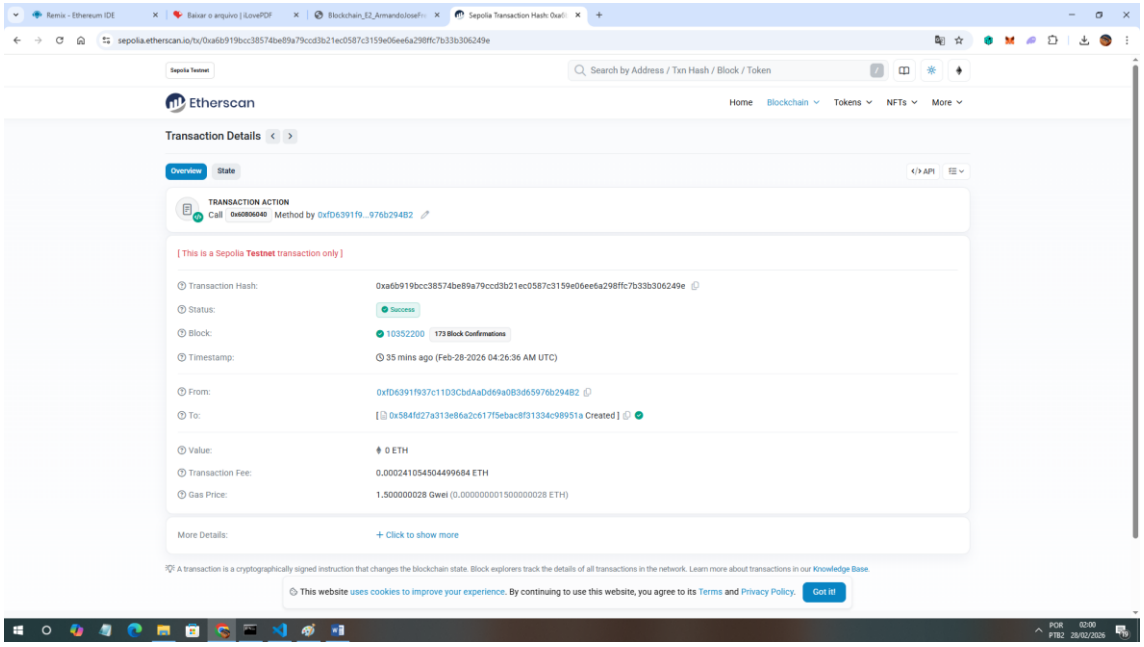
Conclusão 2

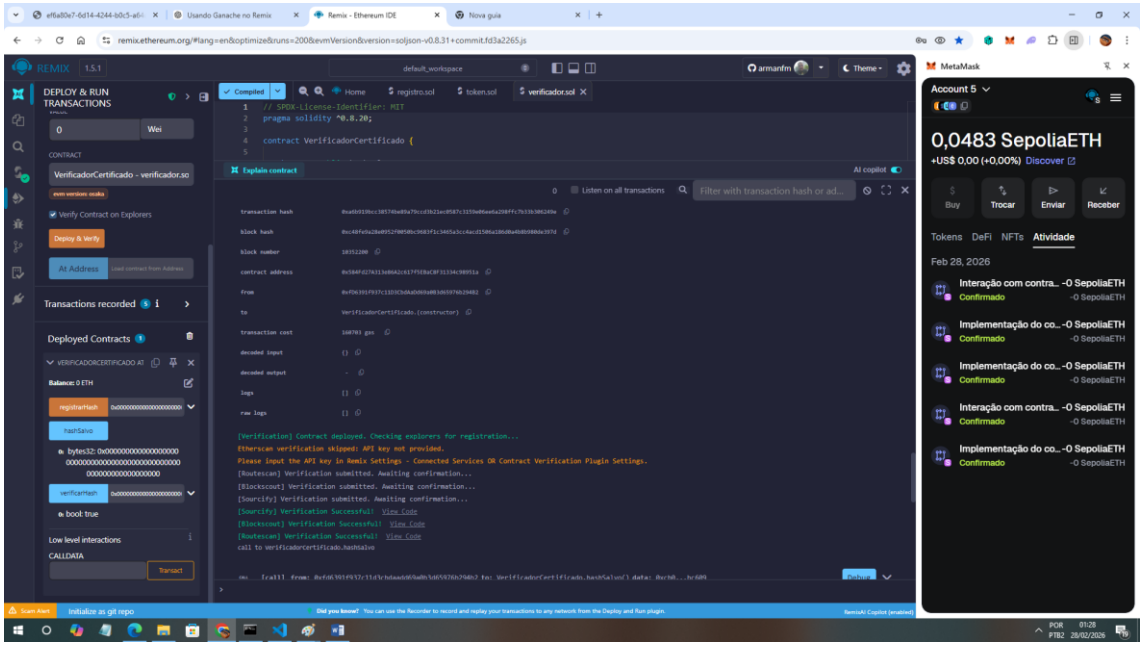
Com isso, o Desafio 2 foi concluído com sucesso: um token simples com supply inicial definido no deploy , transferindo destes tokens para outra carteira usando a função `transfer`. Em seguida, validamos o resultado consultando `totalSupply` (quantidade total criada) e `balanceOf` (saldo por endereço), confirmando que o saldo da carteira emissora diminui e o saldo da carteira receptora aumenta após a transação. Dessa forma, ficou demonstrado o funcionamento básico de emissão, envio e verificação de tokens.

Desafio 3

Verificar um hash de um arquivo

(simulação certificado)





Demonstrar o funcionamento de um contrato inteligente capaz de:

- Registrar um hash na blockchain
- Verificar posteriormente se um determinado hash já foi registrado

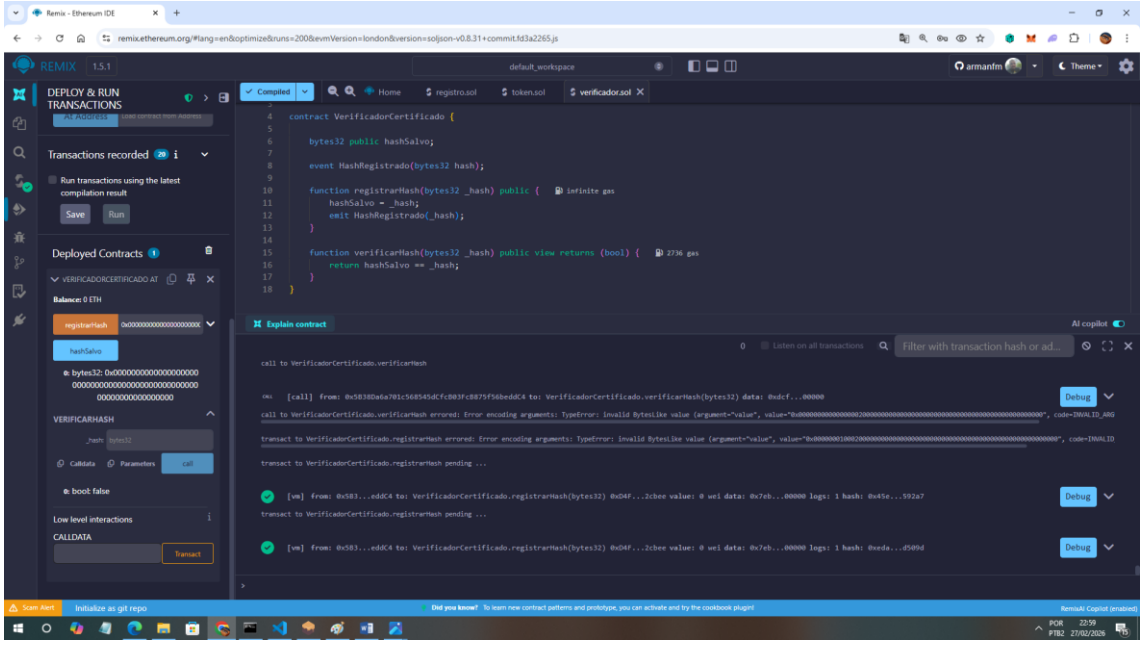
Essa simulação representa, conceitualmente, a validação de um certificado digital através do hash do arquivo.

O contrato `VerificadorCertificado` possui:

- Uma variável `bytes32 public hashSalvo`
- Uma função `registrarHash(bytes32 _hash)`
- Uma função `verificarHash(bytes32 _hash)`

Após compilar o arquivo `verificador.sol`, realizamos o deploy do contrato na rede de teste local.

A função `registrarHash` permite salvar um hash escolhido pelo usuário, enquanto a função `verificarHash` compara o hash informado com o hash previamente salvo.

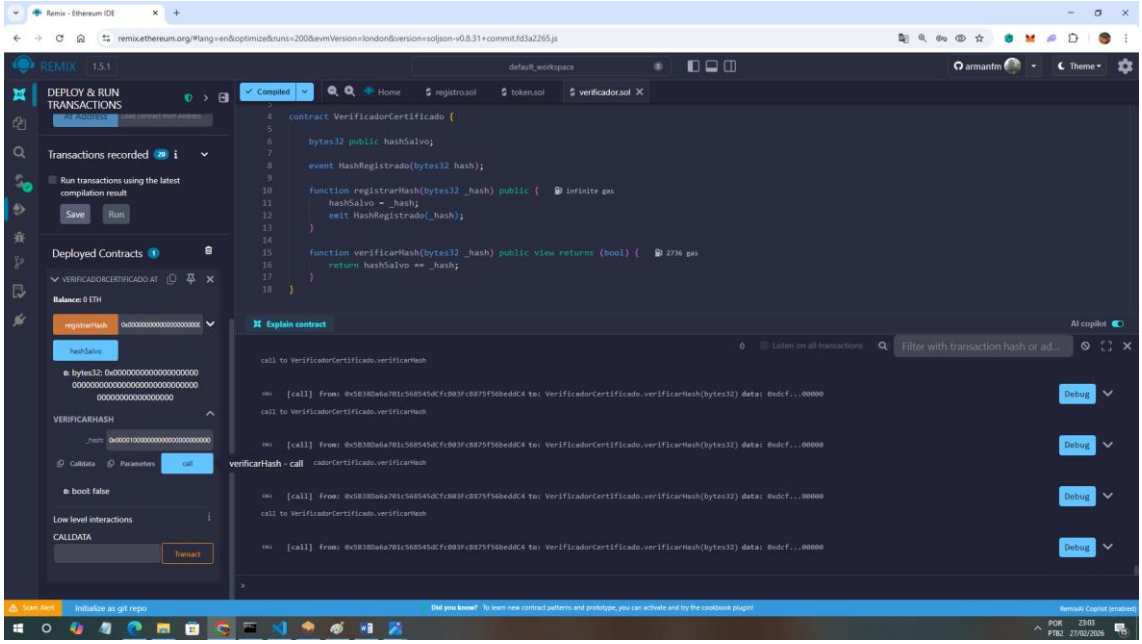


Após o deploy:

1. Registramos um hash (pode ser um valor aleatório ou definido pelo usuário).
2. Utilizamos a função `verificarHash`.
3. Quando o hash informado é exatamente igual ao hash salvo, o sistema retorna:

```
true
```

Isso demonstra que o hash está registrado na blockchain e pode ser validado corretamente.



Por fim, realizamos um teste de validação com um hash diferente do que foi salvo e ao inserir um hash inexistente e executar a verificação, o contrato retorna:

```
false
```

Isso comprova que:

- O sistema compara corretamente os valores.
- A validação ocorre diretamente na blockchain.
- Apenas o hash registrado previamente é considerado válido.

Observação: para fins de demonstração utilzei o Gamach além do testnet.

Conclusão 3

Este contrato demonstra uma aplicação prática da blockchain para validação de integridade de dados, pois ao registrar um hash, garantimos que o dado original pode ser verificado futuramente, a verificação é imutável e a validação ocorre de forma transparente e aditável.